

READ ME

Análisis de Sistema de Producción con IoT

Archivo de datos	Production System Dataset.csv
Script principal	Trabajo.py
Lenguaje	Python 3
Alumna	Florencia Wachlin
DNI	45200881
Numero de alumno	75610/1
Fecha	Junio 2026

1. Descripción de los Datos

Fuente: Kaggle — *Real-Time IoT-Driven Production System Dataset*). El dataset simula datos en tiempo real provenientes de sensores IoT instalados en máquinas de planta industrial. (https://www.kaggle.com/datasets/programmer3/real-time-iot-driven-production-system-dataset?utm_source=chatgpt.com)

Estructura del archivo: Formato CSV, con registros minuto a minuto a partir del 10 de marzo de 2025. Cada fila representa una lectura de un sensor asociado a una máquina específica.

Campo	Tipo	Descripción
timestamp	datetime	Fecha y hora de la lectura
machine_id	string	Identificador único de la máquina (M001–M004)
machine_type	string	Tipo de máquina: Welder, Conveyor, Drill, CNC
temperature	float	Temperatura operativa (°C)
vibration_level	float	Nivel de vibración del equipo
power_consumption	float	Consumo energético (W)
pressure	float	Presión del sistema
material_flow_rate	float	Tasa de flujo de material
cycle_time	float	Tiempo de ciclo de operación (s)
error_rate	float	Tasa de error (1.0 = error crítico)
downtime	int	Tiempo de inactividad (min)
maintenance_flag	int	Indicador de mantenimiento (0/1)
efficiency_score	float	Puntaje de eficiencia operativa (0–100)
production_status	int	Estado de producción (0/1)

2. Análisis Realizados

El script **Trabajo.py** realiza los siguientes análisis sobre el dataset:

- **Exploración inicial:** Se ejecuta `df.info()` y `df.describe()` para conocer la estructura, tipos de datos, presencia de nulos y estadísticos básicos. Esto permite validar la calidad del dataset antes de cualquier procesamiento.
- **Limpieza de datos:** Se verifican valores nulos con `isnull().sum()` y se eliminan registros duplicados con `drop_duplicates()`. Garantiza que los análisis posteriores no estén distorsionados por datos incorrectos o redundantes.
- **Análisis de indicadores operativos por tipo de máquina:** Se calcula el promedio de temperatura, vibración y consumo energético agrupando por `machine_type`. Permite comparar el comportamiento típico de cada tipo de equipo e identificar cuál opera bajo condiciones más exigentes.
- **Conteo de errores críticos por máquina:** Se filtran los registros donde `error_rate == 1` (error crítico) y se cuenta cuántas veces ocurre por tipo de máquina. Sirve para detectar qué equipos presentan mayor frecuencia de fallas y requieren atención prioritaria.
- **Visualizaciones:** Se generan tres gráficos de barras: temperatura y vibración promedio por máquina (`temp_vib.png`), consumo energético (`consumo.png`) y cantidad de errores críticos (`histograma_error.png`). Las imágenes se integran automáticamente en el reporte final.
- **Generación de reporte Word (.docx):** Se exporta un documento `reporte.docx` con los estadísticos por máquina, los gráficos incrustados y una conclusión textual, listo para compartir o presentar.

3. Conclusiones

- Los cuatro tipos de máquinas (Welder, Conveyor, Drill, CNC) presentan niveles de temperatura, vibración y consumo energético relativamente similares entre sí, lo que sugiere un funcionamiento homogéneo en condiciones normales.
- La máquina **Conveyor** concentra la mayor cantidad de registros con `error_rate = 1` (error crítico), destacándose por encima del resto. Esto la convierte en el equipo de mayor riesgo operativo y el candidato prioritario para un análisis de mantenimiento más profundo.
- La presencia de registros con `error_rate = 1` correlaciona con valores de `vibration_level` significativamente más altos (aprox. 7.2–7.4 vs. 2.0–2.2 en operación normal), lo que puede usarse como señal de alerta temprana.

4. Funcionalidades a Implementar Posteriormente

- **Dashboard interactivo:** Migrar las visualizaciones a una herramienta como Power BI o Plotly Dash para explorar los datos dinámicamente con filtros por fecha, máquina o turno.
- **Análisis temporal:** Incorporar la dimensión del tiempo (timestamp) para detectar patrones horarios, por turno o estacionales en el comportamiento de las máquinas.
- **Alertas automáticas:** Implementar un sistema que notifique cuando los promedios de vibración o temperatura superen umbrales definidos.

5. Posibles Mejoras al Código Actual

- **Eliminar `os.chdir()`:** Usar rutas relativas o Path de pathlib en lugar de cambiar el directorio de trabajo, para mayor portabilidad entre sistemas.
- **Modularizar en funciones:** Separar el código en funciones (`cargar_datos()`, `limpiar_datos()`, `analizar()`, `graficar()`, `generar_reporte()`) para mejorar la legibilidad y facilitar el testing.
- **Manejo de excepciones:** Agregar bloques try/except para casos como archivo CSV no encontrado o columnas faltantes, evitando que el programa falle sin explicación.
- **Exportación en múltiples formatos:** Además de .docx, generar el reporte también en PDF directamente desde el script.